

# Program 8: Create Kubernetes Pods using YAML Descriptors

## Aim

To create a Kubernetes **Pod** using **YAML descriptors** and deploy a simple **Node.js application** inside the Pod using a **ConfigMap**.

---

## Prerequisites

- Minikube installed and running
  - kubectl configured
  - Basic knowledge of YAML
- 

## Step 1: Start Minikube

```
minikube start
```

---

## Step 2: Create Node.js Application (server.js)

```
const http = require("http");

const port = 3000;

const server = http.createServer((req, res) => {
  res.end("Hello from Node.js running inside Kubernetes!");
});
```

```
server.listen(port, () => {  
  console.log(`Server running at port ${port}`);  
});
```



The screenshot shows a code editor with a tab labeled "JS server.js". The code is as follows:

```
1  const http=require('http');  
2  const port=3000;  
3  
4  const server=http.createServer((req,res)=>{  
5    |  
6    |  
7    |   res.end('Hello, k8s!\n');  
8  | });  
9  
10 server.listen(port,()=>{  
11 |   console.log(`Server is running on http://localhost:${port}`);  
12 | });
```

## Explanation

- Creates a simple HTTP server
- Listens on port 3000
- Returns a response when accessed

---

## Step 3: Create ConfigMap YAML (nodejs-configmap.yaml)

apiVersion: v1

kind: ConfigMap

metadata:

name: nodejs-app-config

data:

```
server.js: |

const http = require("http");

const port = 3000;


const server = http.createServer((req, res) => {

  res.end("Hello from Node.js running inside Kubernetes!");

});


server.listen(port, () => {

  console.log(`Server running at port ${port}`);

});
```

The screenshot shows the VS Code Explorer with three files: `configMap.yaml`, `pod.yaml`, and `server.js`. The `configMap.yaml` file is selected and its content is displayed in the editor. The content is a Kubernetes ConfigMap definition with the following details:

- `apiVersion: v1`
- `kind: ConfigMap`
- `metadata:`
  - `name: configmap`
- `data:`
  - `server.js:` (containing the JavaScript code from the previous block)

## Explanation

- ConfigMap stores non-sensitive configuration data
- `server.js` is stored inside the ConfigMap
- Allows code to be injected into the Pod without building a Docker image

## Step 4: Create Pod YAML (nodejs-pod.yaml)

apiVersion: v1

kind: Pod

metadata:

name: program-8

spec:

containers:

- name: nodejs

image: node:18

command: ["node", "/usr/src/app/server.js"]

volumeMounts:

- name: app-code

mountPath: /usr/src/app

volumes:

- name: app-code

configMap:

name: nodejs-app-config

! pods.yaml

```
1  apiVersion: v1
2  kind: Pod
3  metadata:
4    | name: my-pod
5  spec:
6    | containers:
7      | - name: node
8        | image: node:14-alpine
9        | command: ["node", "/usr/src/app/server.js"]
10       | volumeMounts:
11         | - name: config-volume
12           | mountPath: /usr/src/app
13     | volumes:
14       | - name: config-volume
15         | configMap:
16           | name: configmap
```

## Explanation

- Pod is the smallest deployable unit in Kubernetes
- Node.js container runs the application
- ConfigMap is mounted as a volume inside the container

---

## Step 5: Apply the YAML Files

kubectl apply -f nodejs-configmap.yaml

kubectl apply -f nodejs-pod.yaml

---

## Step 6: Verify Pod Status

```
kubectl get pods
```

Expected status: **Running**

---

## Step 7: Access the Application

```
kubectl port-forward pod/program-8 3000:3000
```

Open browser and visit:

<http://localhost:3000>

---

## Expected Output

Hello from Node.js running inside Kubernetes!

---

## Conclusion

This experiment demonstrates how to create a Kubernetes Pod using YAML descriptors, use